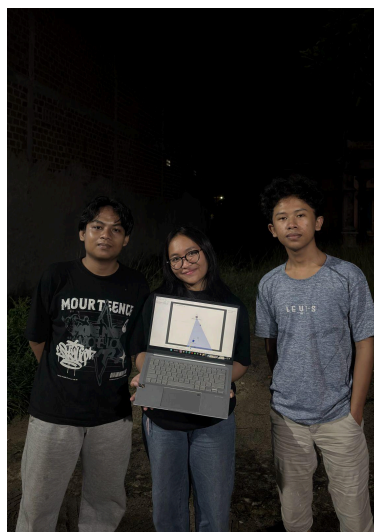


IMPLEMENTASI ALGORITMA GREEDY DALAM PEMECAHAN BOT PERMAINAN DIAMOND

Tugas Besar

Diajukan sebagai syarat menyelesaikan mata kuliah Strategi Algoritma (IF2211) Kelas RC di
Program Studi Teknik Informatika, Fakultas Teknologi Industri,
Institut Teknologi Sumatera



Oleh : Kelompok 9 (GATOT)

Hafidz Haqiqi	124140016
Pray Febry Valentine SG	124140184
Muhamad Rofik Assidiqi	124140010

Dosen Pengampu: Winda Yulita, [M.Cs.](#)

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI SUMATERA
LAMPUNG SELATAN**

2026

Daftar Isi

BAB 1 PENDAHULUAN.....	2
1.1 Latar Belakang.....	2
1.2 Starter Pack.....	5
1.3 Spesifikasi.....	6
1.3.1 Spesifikasi Wajib.....	6
1.3.2 Spesifikasi Bonus.....	6
BAB II LANDASAN TEORI.....	7
2.1 Algoritma Greedy.....	7
2.2 Elemen Algoritma Greedy.....	8
2.3 Game Engine.....	8
2.4 Komponen Program Robocode Tank Royale.....	8
2.5 Cara Menjalankan Bot dan Implementasi Greedy pada Bot.....	9
2.6 Mekanisme Teknis Permainan Robocode Tank Royale.....	10
BAB III APLIKASI STRATEGI GREEDY.....	10
3.1 Mapping Persoalan Robocode Tank Royale ke Elemen Greedy.....	10
3.2 Eksplorasi 4 Alternatif Solusi Greedy.....	11
3.2.1 Alternatif 1: RangeKeeperGreedy — Greedy Ideal Distance.....	11
3.2.2 Alternatif 2: LockBot — Greedy Candidate Position Scoring.....	11
3.2.3 Alternatif 3: AegisGreedy — Greedy Hybrid Target Scoring.....	12
3.2.4 Alternatif 4: MinimumDangerBot — Greedy Minimum Danger.....	12
3.3 Analisis Efisiensi dan Efektivitas Alternatif Greedy.....	12
3.4 Strategi Greedy yang Dipilih.....	13
BAB IV IMPLEMENTASI DAN PENGUJIAN.....	14
4.1 Implementasi 4 Alternatif Solusi.....	14
4.1.1 Pseudocode Bot RangeKeeperGreedy.....	14
4.1.2 Pseudocode Bot LockBot.....	16
4.1.3 Pseudocode Bot AegisGreedy.....	19
4.1.4 Pseudocode Bot MinimumDangerBot.....	23
4.2 Struktur Data, Fungsi, dan Prosedur pada Bot Utama.....	26
4.3 Pengujian.....	27
4.3.1 Pengujian Internal Empat Alternatif Bot.....	27
4.3.2 Pengujian Bot Utama Melawan Bot Asisten.....	28
Pengujian 1:.....	28
4.4 Analisis Hasil Pengujian.....	29
BAB V KESIMPULAN DAN SARAN.....	30
5.1 Kesimpulan.....	30
5.2 Saran.....	30
LAMPIRAN.....	31
Lampiran 1. Tautan Repository GitHub.....	31
Lampiran 2. Video Demo.....	31
DAFTAR PUSTAKA.....	32

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Robocode adalah permainan pemrograman yang bertujuan untuk membuat kode bot dalam bentuk tank virtual untuk berkompetisi melawan bot lain di arena. Pertempuran Robocode berlangsung hingga bot-bot bertarung hanya tersisa satu seperti permainan Battle Royale, karena itulah permainan ini dinamakan Tank Royale. Nama Robocode adalah singkatan dari "Robot code," yang berasal dari versi asli/pertama permainan ini. Robocode Tank Royale adalah evolusi/versi berikutnya dari permainan ini, di mana bot dapat berpartisipasi melalui Internet/jaringan. Dalam permainan ini, pemain berperan sebagai programmer bot dan tidak memiliki kendali langsung atas permainan. Pemain hanya bertugas untuk membuat program yang menentukan logika atau "otak" bot. Program yang dibuat akan berisi instruksi tentang cara bot bergerak, mendeteksi bot lawan, menembakkan senjatanya, serta bagaimana bot bereaksi terhadap berbagai kejadian selama pertempuran.

Pada Tugas Besar pertama Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Tentunya mahasiswa harus menggunakan **strategi greedy** dalam membuat bot ini.

Komponen-komponen dari permainan ini antara lain:

1. Rounds dan Turns

Pertempuran dapat terdiri dari beberapa rounds. Secara default, satu pertempuran berisi 10 rounds, di mana setiap rounds akan memiliki pemenang dan yang kalah. Setiap round dibagi menjadi beberapa turns, yang merupakan unit waktu terkecil. Satu turn adalah satu ketukan waktu dan satu putaran permainan. Jumlah turn dalam satu round tergantung pada berapa lama waktu yang dibutuhkan hingga hanya tersisa bot terakhir yang bertahan.

Pada setiap turn, sebuah bot dapat:

- Menggerakkan bot, memindai musuh, dan menembakkan senjata.
- Bereaksi terhadap peristiwa seperti saat bot terkena peluru atau bertabrakan dengan bot lain atau dinding.
- Perintah untuk bergerak, berputar, memindai, menembak, dan sebagainya dikirim ke server untuk setiap turn.

Perlu diperhatikan bahwa API (Application Programming Interface) bot resmi secara otomatis mengirimkan niat bot ke server di balik layar, sehingga Anda tidak perlu mengkhawatirkannya, kecuali jika Anda membuat API Bot sendiri.

Pada setiap turn, bot akan secara otomatis menerima informasi terbaru tentang posisinya dan orientasinya di medan perang. Bot juga akan mendapatkan informasi tentang bot musuh ketika mereka terdeteksi oleh pemindai.

Perlu diketahui bahwa game engine yang akan digunakan pada tugas besar ini tidak mengikuti aturan default mengenai komponen Round & Turns.

2. Batas Waktu Giliran

Penting untuk dicatat bahwa setiap bot memiliki batas waktu untuk setiap turn yang disebut turn timeout, biasanya antara 30-50 ms (dapat diatur sebagai aturan pertempuran). Ini berarti bahwa bot tidak bisa mengambil waktu sebanyak yang mereka inginkan untuk bergerak dan menyelesaikan turn saat ini.

Setiap kali turn baru dimulai, penghitung waktu ulang diatur ulang dan mulai berjalan. Jika batas waktu tercapai dan bot tidak mengirimkan pergerakannya untuk turn tersebut, maka tidak ada perintah yang dikirim ke server. Akibatnya, bot akan melewatkan turn tersebut. Jika bot melewatkan turn, ia tidak akan bisa menyesuaikan gerakannya atau menembakkan senjatanya karena server tidak menerima perintah tepat waktu sebelum turn berikutnya dimulai.

3. Energi

Semua bot memulai permainan dengan jumlah energi awal sebanyak 100 poin energi.

- Bot akan kehilangan energi jika ditembak atau ditabrak oleh bot musuh.
- Bot juga akan kehilangan energi jika menembakkan meriamnya.
- Bot akan mendapatkan energi jika peluru dari meriamnya mengenai musuh. Energi yang didapat akan lebih banyak 3 kali lipat dari energi yang digunakan untuk menembakkan peluru.
- Bot dengan energi nol akan dinonaktifkan dan tidak bisa bergerak. Jika bot terkena serangan dalam keadaan ini, bot akan hancur.

4. Peluru

Semakin banyak energi (daya tembak) yang digunakan untuk menembakkan peluru, semakin berat peluru tersebut dan semakin lambat gerakannya. Namun, peluru yang lebih berat juga menghasilkan lebih banyak kerusakan dan memungkinkan bot mendapatkan lebih banyak energi saat mengenai bot musuh.

Seperti disebutkan sebelumnya, peluru yang lebih berat akan bergerak lebih lambat. Ini berarti akan membutuhkan waktu lebih lama untuk mencapai target, meningkatkan risiko peluru tidak mengenai sasaran. Sebaliknya, peluru yang lebih ringan bergerak lebih cepat, sehingga lebih mudah mengenai target, tetapi peluru ringan tidak memberikan banyak poin energi saat mengenai bot musuh.

5. Panas Meriam (Gun Heat)

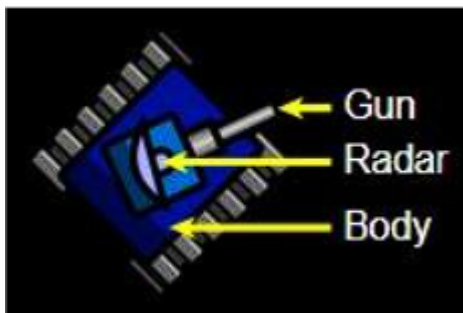
Saat menembakkan peluru, meriam akan menjadi panas. Peluru yang lebih berat menghasilkan lebih banyak panas dibandingkan peluru yang lebih ringan. Ketika meriam terlalu panas, bot tidak dapat menembak hingga suhu meriam turun ke nol. Selain itu, meriam juga sudah dalam keadaan panas di awal round dan perlu waktu untuk mendingin sebelum bisa digunakan untuk pertama kalinya.

6. Tabrakan

Perlu diperhatikan bahwa bot akan menerima kerusakan jika menabrak dinding (batas arena), yang disebut wall damage. Hal yang sama juga terjadi jika bot bertabrakan dengan bot lain. Jika bot menabrak bot musuh dengan bergerak maju, ini disebut ramming (menabrak dengan sengaja), yang akan memberikan sedikit skor tambahan bagi bot yang menyerang.

7. Bagian Tubuh Tank

Tubuh tank terdiri dari 3 bagian:



- *Body* adalah bagian utama dari tank yang digunakan untuk menggerakkan tank.
- *Gun* digunakan untuk menembakkan peluru dan dapat berputar bersama *body* atau independen dari *body*.
- *Radar* digunakan untuk memindai posisi musuh dan dapat berputar bersama *body* atau independen dari *body*.

8. Pergerakan

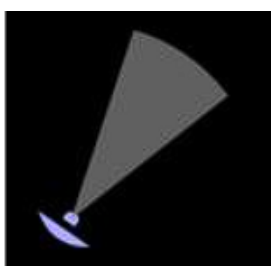
Bot dapat bergerak maju dan mundur hingga kecepatan maksimum. Dibutuhkan beberapa giliran untuk mencapai kecepatan maksimum. Bot dapat mengalami percepatan maksimum sebesar 1 unit per giliran dan pengereman dengan perlambatan maksimum 2 unit per giliran. Percepatan dan perlambatan maksimum tidak bergantung pada kecepatan bot saat itu.

9. Berbelok

Seperti yang disebutkan sebelumnya, bagian tubuh, turret (meriam), dan radar dapat berputar secara independen satu sama lain. Jika turret atau radar tidak diputar, maka keduanya akan mengarah ke arah yang sama dengan tubuh bot. Setiap bagian tubuh memiliki kecepatan putar yang berbeda. Radar adalah bagian tercepat dan dapat berputar hingga 45 derajat per giliran, yang berarti dapat berputar 360 derajat dalam 8 giliran. Turret dan meriam dapat berputar hingga 20 derajat per giliran. Bagian paling lambat adalah tubuh tank, yang dalam kondisi terbaik dapat berputar hingga 10 derajat per giliran. Namun, ini bergantung pada kecepatan bot saat ini. Semakin cepat bot bergerak, semakin lambat kemampuannya untuk berbelok. Perlu diperhatikan bahwa tidak ada energi yang dikonsumsi saat bot bergerak atau berbelok.

10. Pemindaian

Aspek penting dalam Robocode adalah memindai bot musuh menggunakan radar. Radar dapat mendeteksi bot dalam jangkauan hingga 1200 piksel. Musuh yang berada lebih dari 1200 piksel dari bot tidak dapat terdeteksi atau dipindai oleh radar. Penting untuk diperhatikan bahwa sebuah bot hanya dapat memindai bot musuh yang berada dalam jangkauan sudut pemindaian (scan arc)-nya. Sudut pemindaian ini merupakan "sapuan radar" dari arah radar sebelumnya ke arah radar saat ini dalam satu giliran.



Jika radar tidak bergerak dalam suatu giliran, artinya radar tetap mengarah ke arah yang sama seperti pada giliran sebelumnya, maka sudut pemindaian akan menjadi nol derajat, dan bot tidak akan dapat mendeteksi musuh.



Oleh karena itu, sangat disarankan untuk selalu mengubah arah radar agar tetap dapat memindai musuh.

11. Skor

Pada akhir pertempuran, setiap bot akan diranking berdasarkan total skor yang diperoleh masing-masing bot selama keseluruhan pertempuran. Tentunya, tujuan utama pada tugas besar ini adalah membuat bot yang memberikan skor setinggi mungkin. Berikut adalah rincian komponen skor pada pertempuran:

- **Bullet Damage:** Bot mendapatkan poin sebesar *damage* yang dibuat kepada bot musuh menggunakan peluru.
- **Bullet Damage Bonus:** Apabila peluru berhasil membunuh bot musuh, bot mendapatkan poin sebesar 20% dari *damage* yang dibuat kepada musuh yang terbunuh.
- **Survival Score:** Setiap ada bot yang mati, bot lain yang masih bertahan pada ronde tersebut mendapatkan 50 poin.
- **Last Survival Bonus:** Bot terakhir yang bertahan pada suatu ronde akan mendapatkan 10 poin dikali dengan banyaknya musuh.
- **Ram Damage:** Bot mendapatkan poin sebesar 2 kalinya *damage* yang dibuat kepada bot musuh dengan cara menabrak.
- **Ram Damage Bonus:** Apabila musuh terbunuh dengan cara ditabrak, bot mendapatkan poin sebesar 30% dari *damage* yang dibuat kepada musuh yang terbunuh.

Skor akhir bot adalah akumulasi dari 6 komponen diatas. Perlu diperhatikan bahwa game akan menampilkan berapa kali suatu bot meraih peringkat 1, 2, atau 3 pada setiap ronde. Namun, hal ini tidak dihitung sebagai komponen skor maupun untuk perangkingan akhir. Bot yang dianggap menang pertempuran adalah bot dengan akumulasi skor tertinggi.

1.2 Starter Pack

Untuk tugas besar ini, game engine yang akan digunakan sudah dimodifikasi oleh asisten. Berikut adalah beberapa perubahan yang dibuat oleh asisten dari game engine Tank Royale:

- **Theme GUI:** diubah menjadi light theme.
- **Skor & Energi:** masing-masing bot ditampilkan di samping arena permainan agar lebih mudah diamati saat pertarungan.

- Turn Limit: Durasi pertarungan tidak akan bergantung pada banyak ronde. Pada game engine ini, pertarungan akan berakhir apabila banyaknya turn sudah mencapai batas tertentu. Apabila batasan ini tercapai, ronde otomatis langsung berakhir dan pemenang pertarungan akan ditampilkan. Turn Limit dapat diatur pada menu “setup rules”.

1.3 Spesifikasi

Pada tugas ini kami diminta untuk membuat program sederhana dalam bahasa C# yang mengimplementasikan algoritma Greedy pada bot permainan Robocode Tank Royale dengan tujuan memenangkan permainan. Program perlu dibuat dengan spesifikasi sebagai berikut.

1.3.1 Spesifikasi Wajib

1. Buatlah 4 bot (1 utama dan 3 alternatif) dalam bahasa C# (.net) yang mengimplementasikan *algoritma Greedy* pada bot permainan Robocode Tank Royale dengan tujuan memenangkan permainan.
2. Tugas dikerjakan berkelompok dengan anggota minimal 2 orang dan maksimal 3 orang, boleh lintas kelas dan lintas kampus.
3. Strategi greedy yang diimplementasikan setiap kelompok harus dikaitkan dengan fungsi objektif dari permainan ini, yaitu memperoleh skor setinggi mungkin pada akhir pertempuran. Hal ini dapat dilakukan dengan mengoptimalkan komponen skor yang telah dijelaskan diatas.
4. Strategi greedy yang diimplementasikan harus berbeda untuk setiap bot yang diimplementasikan dan setiap strategi greedy harus menggunakan heuristic yang berbeda.
5. Bot yang dibuat TIDAK BOLEH sama dengan SAMPEL yang diberikan sebagai CONTOH. Baik dari starter pack maupun dari repository engine asli.
6. Buatlah strategi greedy terbaik, karena setiap bot utama dari masing-masing kelompok akan diadu dalam kompetisi Tubes 1.
7. Strategi greedy yang kelompok anda buat harus dijelaskan dan ditulis secara eksplisit pada laporan, karena akan diperiksa saat demo apakah strategi yang dituliskan sesuai dengan yang diimplementasikan.
8. Setiap kelompok dapat menggunakan kreativitas yang bermacam macam dalam menyusun strategi greedy untuk memenangkan permainan. Implementasi pemain harus dapat dijalankan pada game engine yang telah disebutkan diatas serta dapat dikompetisikan dengan bot dari kelompok lain.
9. Program harus mengandung komentar yang jelas, dan untuk setiap strategi greedy yang disebutkan, harus dilengkapi dengan kode sumber yang dibuat. Artinya semua strategi harus diimplementasikan.
10. Mahasiswa disarankan membaca dokumentasi dari game engine. Perlu diperhatikan bahwa game engine yang digunakan untuk tubes ini SUDAH DIMODIFIKASI. Untuk Perubahan dapat dilihat pada bagian starter pack.

1.3.2 Spesifikasi Bonus

1. (maks 10) Membuat video tentang aplikasi greedy pada bot serta simulasinya pada game kemudian mengunggahnya di Youtube. Video dibuat harus memiliki audio dan menampilkan wajah dari setiap anggota kelompok. Untuk contoh video tubes stima tahun-tahun sebelumnya dapat dilihat di

Youtube dengan kata kunci “Tubes Stima”, “strategi algoritma”, “Tugas besar stima”, dll. Semakin menarik video, maka semakin banyak poin yang diberikan.

2. (maks 10) Beberapa kelompok pemenang lomba kompetisi akan mendapatkan nilai tambahan berdasarkan posisi yang diraih.

BAB II LANDASAN TEORI

2.1 Algoritma Greedy

Algoritma greedy merupakan salah satu algoritma yang digunakan untuk menyelesaikan permasalahan optimasi. Algoritma ini bekerja dengan cara memilih keputusan terbaik pada setiap langkah berdasarkan kondisi yang sedang terjadi saat itu. Keputusan yang dipilih disebut sebagai keputusan lokal terbaik, dengan harapan dapat menghasilkan solusi akhir yang baik.

Dalam algoritma greedy, sistem tidak mengecek seluruh kemungkinan solusi dari awal sampai akhir, tetapi langsung memilih pilihan yang dianggap paling menguntungkan pada kondisi saat ini. Oleh karena itu, algoritma greedy memiliki proses yang cepat dan cocok digunakan pada sistem yang membutuhkan pengambilan keputusan secara real-time.

Pada permainan Robocode Tank Royale, algoritma greedy cocok digunakan karena bot harus mengambil keputusan secara cepat pada setiap tick permainan. Bot harus menentukan arah gerak, memilih target, mengatur radar, menentukan kekuatan tembakan, serta menghindari serangan musuh dalam waktu singkat.

2.2 Elemen Algoritma Greedy

Algoritma greedy memiliki beberapa elemen penting, yaitu:

Elemen Greedy	Penjelasan
Himpunan Kandidat	Kumpulan pilihan yang dapat dipilih oleh bot, seperti posisi tujuan, target musuh, arah radar, dan kekuatan tembakan
Himpunan Solusi	Pilihan yang sudah dipilih sebagai keputusan bot
Fungsi Seleksi	Fungsi untuk memilih kandidat terbaik berdasarkan heuristic tertentu
Fungsi Kelayakan	Fungsi untuk memastikan pilihan dapat dijalankan, misalnya posisi tidak keluar arena atau tidak terlalu dekat dinding
Fungsi Objektif	Tujuan yang ingin dicapai, yaitu memperoleh skor setinggi mungkin

Dalam tugas ini, fungsi objektif dari bot adalah memperoleh skor akhir setinggi mungkin. Hal tersebut dilakukan dengan meningkatkan kemampuan bertahan hidup, menghindari peluru, memberikan damage kepada musuh, dan memenangkan pertandingan.

2.3 Game Engine

Robocode Tank Royale adalah game engine yang digunakan dalam tugas, merupakan evolusi dari Robocode asli di mana pemain membuat program bot tank virtual untuk berkompetisi satu sama lain. Untuk tugas besar ini, game engine tersebut telah dimodifikasi oleh asisten dengan perubahan berupa

tampilan light theme, tampilan skor dan energi bot di samping arena, serta perubahan aturan pertarungan yang tidak lagi bergantung pada jumlah ronde melainkan jumlah turn yang dapat diatur batasnya. Pemain tidak memiliki kendali langsung selama permainan, hanya bertugas membuat program C# (.NET) dengan algoritma Greedy yang menentukan logika bot dalam bergerak, mendeteksi lawan, menembak, dan bereaksi terhadap kejadian selama pertempuran untuk mendapatkan skor tertinggi.

2.4 Komponen Program Robocode Tank Royale

Program ini terdiri atas *game engine* dan *bot stater pack*. Kedua komponen tersebut tersedia pada <https://github.com/Ariel-HS/tubes1-if2211-starter-pack>. Untuk menjalankan game engine digunakan `robocode-tankroyale-gui-0.30.0.jar`. Dalam *repository* yang sama tersedia *starter pack* lain yang dapat digunakan untuk menjalankan program. Sebagai contoh, tersedia file "TemplateBot.zip" yang menyediakan template dasar untuk membuat bot C# dengan struktur file yang diperlukan, termasuk kerangka kode bot, file konfigurasi JSON, dan file build (`.csproj`, `.cmd`, `.sh`).

2.5 Cara Menjalankan Bot dan Implementasi Greedy pada Bot

Untuk menjalankan bot:

1. Jalankan game engine, dengan perintah `java -jar robocode-tankroyale-gui-0.30.0.jar` pada cmd.



2. Setup konfigurasi bot root directory. Dimulai dengan klik tombol "Config", lalu klik tombol "Bot Root Directories", kemudian masukkan *directory* yang berisi folder-folder bot terkait.
3. Mulai battle. Dimulai dengan klik tombol "Battle", lalu klik "Start Battle". Nantinya akan tersedia bot-bot di dalam *directory* yang telah disetup.
4. Boot bot yang ingin dimainkan. Dimulai dengan memilih bot yang tersedia pada bagian kotak kiri-atas, lalu klik tombol "Boot". Selanjutnya bot yang telah dipilih akan muncul pada kotak kanan-atas, bot yang berhasil di boot akan muncul pada kotak kiri-bawah. Terakhir tambahkan

semua bot dalam permainan dengan menekan tombol “Add”, lalu mulai permainan dengan klik “Start Battle”.

Implementasi bot menggunakan bahasa C# dengan kerangka minimal berisi class yang mewarisi Bot, konstruktor yang memuat file JSON, dan metode Run(). Implementasi algoritma Greedy pada bot dapat dilakukan dengan mengoptimalkan keputusan setiap turn berdasarkan kondisi terkini seperti posisi musuh, energi tersisa, dan status gun heat untuk memaksimalkan skor dengan komponen-komponen seperti Bullet Damage, Survival Score, Ram Damage, dan lain sebagainya.

2.6 Mekanisme Teknis Permainan Robocode Tank Royale

Setiap bot memulai dengan 100 poin energi yang berkurang saat menembak atau terkena serangan dan bertambah saat peluru mengenai musuh (3x lipat energi yang digunakan). Peluru berat bergerak lebih lambat tapi menghasilkan damage lebih besar. Setelah menembak, meriam menjadi panas dan perlu waktu untuk dingin sebelum dapat menembak lagi. Bot menerima damage saat menabrak dinding atau bot lain. Pergerakan bot memiliki percepatan maksimum 1 unit dan perlambatan maksimum 2 unit per giliran. Pemindaian musuh dilakukan dengan radar yang memiliki jangkauan hingga 1200 piksel dan efektif saat bergerak. Setup permainan lebih lanjut akan disesuaikan pada waktu demo.

BAB III APLIKASI STRATEGI GREEDY

3.1 Mapping Persoalan Robocode Tank Royale ke Elemen Greedy

Persoalan dalam Robocode Tank Royale dapat dimapping ke dalam elemen algoritma greedy sebagai berikut:

Elemen Greedy	Penerapan pada Robocode Tank Royale
Himpunan Kandidat	Target musuh, arah gerak, posisi tujuan, kekuatan peluru, arah radar
Himpunan Solusi	Target yang dipilih, posisi movement yang dipilih, fire power yang digunakan
Fungsi Solusi	Bot berhasil memilih aksi yang dapat dijalankan pada tick tersebut
Fungsi Seleksi	Memilih kandidat dengan skor terbaik berdasarkan heuristic
Fungsi Kelayakan	Posisi tidak keluar arena, tidak terlalu dekat dinding, energi cukup untuk menembak
Fungsi Objektif	Memperoleh skor setinggi mungkin dengan meningkatkan damage, survival, dan kill

3.2 Eksplorasi 4 Alternatif Solusi Greedy

3.2.1 Alternatif 1: RangeKeeperGreedy — Greedy Ideal Distance

RangeKeeperGreedy menggunakan strategi greedy berdasarkan jarak ideal terhadap musuh. Bot akan mendekat jika musuh terlalu jauh, menjauh jika musuh terlalu dekat, dan melakukan strafe jika berada pada jarak ideal. Strategi ini bertujuan menjaga bot tetap berada pada posisi tembak yang efektif.

Heuristic yang digunakan adalah:

- jika jarak musuh lebih besar dari batas maksimum, bot mendekat;
- jika jarak musuh lebih kecil dari batas minimum, bot menjauh;
- jika jarak berada pada area ideal, bot melakukan strafe;
- jika mendeteksi musuh menembak melalui penurunan energi, bot melakukan dodge.

Kode RangeKeeperGreedy menunjukkan penggunaan `MIN_DISTANCE`, `IDEAL_DISTANCE`, dan `MAX_DISTANCE` untuk menentukan apakah bot harus mendekat, menjauh, atau melakukan strafe.

3.2.2 Alternatif 2: LockBot — Greedy Candidate Position Scoring

LockBot menggunakan strategi greedy dengan mengevaluasi beberapa kandidat posisi. Bot membuat beberapa kemungkinan arah dan jarak gerak, kemudian menghitung skor dari setiap kandidat. Posisi dengan skor tertinggi dipilih sebagai posisi tujuan.

Heuristic yang digunakan adalah:

- menjaga jarak ideal dari musuh;
- memaksimalkan gerakan lateral agar sulit terkena peluru;
- menghindari posisi dekat dinding;
- menghindari pojok arena;
- menghindari duel dekat ketika energi rendah.

LockBot memiliki fungsi `ChooseGreedyMove()` dan `ScorePosition()` untuk mengevaluasi kandidat posisi. Fungsi tersebut memberikan penalti pada posisi berbahaya dan bonus pada posisi yang mendukung gerakan lateral serta jarak ideal.

3.2.3 Alternatif 3: AegisGreedy — Greedy Hybrid Target Scoring

AegisGreedy menggunakan strategi greedy hybrid. Bot tidak hanya memilih movement, tetapi juga memilih target terbaik berdasarkan skor. Target yang dipilih adalah musuh dengan kombinasi terbaik dari jarak, energi, kesegaran data scan, dan stabilitas target.

Heuristic yang digunakan adalah:

- memilih musuh yang lebih dekat;
- memilih musuh dengan energi lebih rendah;
- memilih musuh dengan data scan terbaru;
- mempertahankan target agar tidak sering berganti;
- mengejar posisi terakhir musuh jika target hilang;
- membedakan strategi movement antara mode duel dan multi-bot.

AegisGreedy memiliki fungsi `SelectBestTarget()` yang menghitung skor target menggunakan `distanceScore`, `weakScore`, dan `freshScore`. Bot juga memberikan bonus pada target yang sedang dikunci agar target tidak terlalu sering berganti.

3.2.4 Alternatif 4: MinimumDangerBot — Greedy Minimum Danger

MinimumDangerBot menggunakan strategi greedy berdasarkan nilai bahaya posisi. Bot mengevaluasi beberapa kandidat posisi di sekitar bot, kemudian memilih posisi dengan nilai bahaya paling kecil. Strategi ini lebih berfokus pada survival dibanding serangan langsung.

Heuristic yang digunakan adalah:

- memilih posisi dengan nilai bahaya paling rendah;
- menjauhi musuh yang berbahaya;
- menghindari lintasan peluru virtual;
- menghindari dinding dan pojok arena;
- menggunakan data gerakan musuh untuk prediksi.

Kode Woff/MinimumDangerBot menunjukkan penggunaan daftar peluru virtual, data musuh, dan fungsi `CalcGrav()` untuk menghitung nilai bahaya pada kandidat posisi. Bot memilih posisi dengan nilai gravitasi paling kecil sebagai tujuan movement.

3.3 Analisis Efisiensi dan Efektivitas Alternatif Greedy

Alternatif	Efisiensi	Efektivitas	Kelebihan	Kekurangan
RangeKeeperGreedy	Tinggi	Cukup baik untuk 1 vs 1	Sederhana dan cepat	Kurang kuat untuk multi-bot
LockBot	Sedang	Baik untuk duel	Movement lebih terarah	Kandidat posisi masih terbatas
AegisGreedy	Sedang	Baik untuk 1 vs 1 dan multi-bot	Seimbang antara serangan dan bertahan	Parameter lebih kompleks
MinimumDangerBot	Lebih berat	Baik untuk survival	Mampu memilih posisi aman	Perhitungan lebih banyak

RangeKeeperGreedy memiliki efisiensi tinggi karena hanya mengambil keputusan berdasarkan jarak. Namun, strategi ini kurang efektif ketika menghadapi banyak musuh. LockBot lebih efektif karena menggunakan scoring posisi, tetapi masih terbatas pada kandidat posisi tertentu. AegisGreedy lebih seimbang karena menggabungkan target scoring, radar lock, movement adaptif, dan pengaturan energi. MinimumDangerBot memiliki survival tinggi, tetapi lebih berat karena harus menghitung nilai bahaya dari banyak kandidat posisi.

3.4 Strategi Greedy yang Dipilih

Strategi greedy yang dipilih sebagai bot utama adalah **AegisGreedy**. Bot ini dipilih karena memiliki kombinasi strategi yang paling seimbang dibandingkan alternatif lain. AegisGreedy tidak hanya fokus menyerang, tetapi juga mempertimbangkan stabilitas target, efisiensi energi, radar lock, movement adaptif, dan kondisi multi-bot.

Alasan pemilihan AegisGreedy adalah sebagai berikut:

1. Dapat digunakan pada pertandingan 1 vs 1 maupun multi-bot.
2. Memiliki sistem target scoring yang lebih stabil.
3. Tidak mudah kehilangan musuh pada jarak jauh.
4. Mampu mengejar posisi terakhir target ketika radar kehilangan musuh.
5. Mengatur fire power agar energi tidak cepat habis.
6. Memiliki movement berbeda untuk duel dan multi-bot.

Dengan pertimbangan tersebut, AegisGreedy dipilih sebagai bot utama yang akan dikumpulkan dan digunakan dalam kompetisi.

BAB IV IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi 4 Alternatif Solusi

4.1.1 Pseudocode Bot RangeKeeperGreedy

Strategi: Greedy Ideal Distance

```
class RangeKeeperGreedy
{
    InitializeBot()
    {
        Set warna bot;
        Aktifkan radar berputar;
        Matikan fire assist agar tembakan dapat dikontrol manual;
    }

    Run()
    {
        while (bot masih berjalan)
        {
            if (target belum ditemukan || data target sudah lama)
            {
                SearchMovement();
            }
            else
            {
                distance = HitungJarakKeTarget();

                if (bot dekat dinding)
                {
                    MoveToCenter();
                }
                else if (musuh baru menembak)
                {
                    DodgeMovement();
                }
                else if (distance > MAX_DISTANCE)
                {
                    MoveTowardTarget();
                }
            }
        }
    }
}
```

```

        else if (distance < MIN_DISTANCE)
        {
            MoveAwayFromTarget();
        }
        else
        {
            StrafeAroundTarget();
        }
    }

    JalankanAksiBot();
}

}

OnScannedBot(enemy)
{
    Simpan posisi musuh;
    Simpan energi musuh;
    Simpan arah dan kecepatan musuh;

    LockRadarKeMusuh();

    if (EnergiMusuhTurun())
    {
        UbahArahGerak();
        AktifkanDodge();
    }

    firePower = TentukanFirePowerBerdasarkanJarakDanEnergi();
    posisiPrediksi = PrediksiPosisiMusuh(firePower);

    ArahkanGunKe(posisiPrediksi);

    if (gun sudah mengarah && energi cukup)
    {
        Tembak(firePower);
    }
}

MoveTowardTarget()

```



```

{
    // Greedy memilih aksi mendekat jika musuh terlalu jauh
    ArahkanBotKeMusuhDenganSudutMiring();
    Maju();
}

MoveAwayFromTarget()
{
    // Greedy memilih aksi menjauh jika musuh terlalu dekat
    ArahkanBotMenjauhDariMusuh();
    Mundur();
}

StrafeAroundTarget()
{
    // Greedy menjaga jarak ideal sambil bergerak menyamping
    BergerakMenyampingMengelilingiMusuh();
}

DodgeMovement()
{
    // Greedy menghindari peluru ketika musuh diduga menembak
    UbahArahGerakSecaraCepat();
    BergerakMenyamping();
}
}

```

4.1.2 Pseudocode Bot LockBot

Strategi: Greedy Candidate Position Scoring

```

class LockBot
{
    InitializeBot()
    {
        Set warna bot;
        Aktifkan radar;
        Siapkan variabel target;
        Siapkan variabel energi musuh;
    }
}

```

```

Run()
{
    while (bot masih berjalan)
    {
        hasTarget = CekApakahTargetMasihValid();

        if (bot dekat dinding)
        {
            WallEscapeMove();
        }
        else if (!hasTarget)
        {
            SearchTarget();
        }
        else
        {
            LockRadar();
            ChooseGreedyMove();
        }

        JalankanAksiBot();
    }
}

```

```

OnScannedBot(enemy)
{
    Simpan posisi target sebelumnya;
    Simpan posisi target sekarang;

    UpdateEnemyMovement();
    DetectEnemyFire();

    distance = HitungJarakKeTarget();

    LockRadar();
    AimAndFire(distance);
}

```

```

ChooseGreedyMove()

```

```

{
    bestScore = nilai sangat kecil;
    bestAngle = arah saat ini;
    bestDistance = jarak awal;

    daftarSudut = {-145, -120, -95, -75, -55, 55, 75, 95, 120, 145};
    daftarJarak = {90, 120, 150, 180};

    foreach (sudut in daftarSudut)
    {
        foreach (jarak in daftarJarak)
        {
            kandidatPosisi = HitungPosisiBerikutnya(sudut, jarak);

            if (kandidatPosisi layak digunakan)
            {
                score = ScorePosition(kandidatPosisi);

                if (score > bestScore)
                {
                    bestScore = score;
                    bestAngle = sudut;
                    bestDistance = jarak;
                }
            }
        }
    }

    BergerakKeArah(bestAngle, bestDistance);
}

ScorePosition(posisi)
{
    score = 0;

    score += NilaiJarakIdealTerhadapMusuh();
    score += NilaiGerakanLateral();
    score += NilaiJarakDariDinding();
    score -= PenaltiJikaDekatPojoek();
    score -= PenaltiJikaEnergiRendahDanTerlaluDekatMusuh();
}

```

```

    return score;
}

AimAndFire(distance)
{
    firePower = TentukanFirePower(distance);
    posisiPrediksi = PrediksiPosisiMusuh(firePower);

    ArahkanGunKe(posisiPrediksi);

    if (gun terkunci && cooldown siap && energi cukup)
    {
        Tembak(firePower);
    }
}
}

```

4.1.3 Pseudocode Bot AegisGreedy

Strategi: Greedy Hybrid Target Scoring

```

class AegisGreedy
{
    InitializeBot()
    {
        Set warna bot;
        Matikan fire assist;
        Aktifkan radar berputar;
        Siapkan dictionary untuk menyimpan data musuh;
    }

    Run()
    {
        while (bot masih berjalan)
        {
            UpdateTick();

            SelectBestTarget();

```

```

    if (tidak ada target)
    {
        SearchMove();
    }
    else
    {
        if (jumlah musuh terdeteksi > 1)
        {
            MultiWaveMovement();
        }
        else
        {
            DuelMovement();
        }

        RadarControl();
        AimAndFire();
    }

    JalankanAksiBot();
}

OnScannedBot(enemy)
{
    UpdateEnemyData(enemy);

    SelectBestTarget();

    if (enemy adalah target utama)
    {
        if (EnergiMusuhTurun())
        {
            AktifkanDodge();
            UbahArahGerak();
        }

        RadarControl();
        AimAndFire();
    }
}

```

```

}

SelectBestTarget()
{
    bestScore = nilai sangat kecil;
    bestTarget = kosong;

    foreach (enemy in daftarMusuh)
    {
        if (enemy sudah mati)
        {
            continue;
        }

        distanceScore = HitungSkorBerdasarkanJarak(enemy);
        weakScore = HitungSkorBerdasarkanEnergiRendah(enemy);
        freshScore = HitungSkorBerdasarkanDataScanTerbaru(enemy);

        score = 0;
        score += distanceScore;
        score += weakScore;
        score += freshScore;

        if (enemy adalah target saat ini)
        {
            score += bonusStabilitasTarget;
        }

        if (enemy berada pada jarak efektif tembakan)
        {
            score += bonusJarakEfektif;
        }

        if (score > bestScore)
        {
            bestScore = score;
            bestTarget = enemy;
        }
    }
}

```

```

    targetUtama = bestTarget;
}

DuelMovement()
{
    distance = HitungJarakKeTarget();

    if (bot dekat dinding)
    {
        WallEscape();
    }
    else if (target jauh)
    {
        DekatiTargetDenganSudutMiring();
    }
    else if (target terlalu dekat)
    {
        MenjauhDariTarget();
    }
    else
    {
        OrbitMengelilingiTarget();
    }
}

MultiWaveMovement()
{
    // Digunakan saat ada lebih dari satu musuh
    gayaX = 0;
    gayaY = 0;

    TambahkanGayaMenjauhDariDinding();

    foreach (enemy in daftarMusuh)
    {
        TambahkanGayaMenjauhDariMusuh(enemy);
    }

    TambahkanGayaMendekatiTargetUtamaJikaTerlaluJauh();
    TambahkanGayaOrbitTerhadapTargetUtama();
}

```

```

posisiTujuan = HitungPosisiBerdasarkanGaya(gayaX, gayaY);

BergerakKe(posisiTujuan);
}

RadarControl()
{
    if (tidak ada target)
    {
        PutarRadarMencariMusuh();
    }
    else if (target sudah lama tidak terlihat)
    {
        ArahkanRadarKePosisiTerakhirTarget();
    }
    else
    {
        KunciRadarKeTarget();
    }
}

AimAndFire()
{
    if (data target terlalu lama && target jauh)
    {
        JanganMenembak();
        return;
    }

    firePower = TentukanFirePower();

    posisiPrediksi = PrediksiPosisiMusuh(firePower);

    ArahkanGunKe(posisiPrediksi);

    if (gun akurat && energi cukup && gun tidak panas)
    {
        Tembak(firePower);
    }
}

```



```
}  
}
```

4.1.4 Pseudocode Bot MinimumDangerBot

Strategi: Greedy Minimum Danger

```
class MinimumDangerBot  
{  
    InitializeBot()  
    {  
        Set warna bot;  
        Aktifkan radar berputar;  
        Siapkan data musuh;  
        Siapkan daftar virtual bullet;  
        Siapkan posisi tujuan awal;  
    }  
  
    Run()  
    {  
        while (bot masih berjalan)  
        {  
            UpdateVirtualBullet();  
  
            bestPosition = posisi saat ini;  
            lowestDanger = nilai sangat besar;  
  
            daftarKandidat = BuatKandidatPosisiDiSekitarBot();  
  
            foreach (posisi in daftarKandidat)  
            {  
                if (PosisiTidakLayak(posisi))  
                {  
                    continue;  
                }  
  
                danger = CalculateDanger(posisi);  
  
                if (danger < lowestDanger)  
                {
```

```

        lowestDanger = danger;
        bestPosition = posisi;
    }
}

BergerakKe(bestPosition);

JalankanAksiBot();
}
}

OnScannedBot(enemy)
{
    SimpanDataMusuh(enemy);

    if (enemy lebih dekat dari target saat ini)
    {
        targetUtama = enemy;
    }

    LockRadarKeTarget();

    if (EnergiMusuhTurun())
    {
        TambahkanVirtualBullet(enemy);
    }

    SimpanPolaGerakanMusuh(enemy);

    firePower = TentukanFirePower(enemy);
    posisiPrediksi = PrediksiGerakanMusuh(enemy, firePower);

    ArahkanGunKe(posisiPrediksi);

    if (gun sudah mengarah && energi cukup)
    {
        Tembak(firePower);
    }
}

```

```

UpdateVirtualBullet()
{
    foreach (bullet in daftarVirtualBullet)
    {
        GerakkanBulletSesuaiArahDanKecepatan();

        if (bullet keluar arena)
        {
            HapusBullet();
        }
    }
}

BuatKandidatPosisiDiSekitarBot()
{
    daftarKandidat = kosong;

    for (setiap sudut di sekitar bot)
    {
        for (setiap radius gerak)
        {
            posisi = HitungPosisiDariSudutDanRadius();
            Tambahkan posisi ke daftarKandidat;
        }
    }

    return daftarKandidat;
}

CalculateDanger(posisi)
{
    danger = 0;

    foreach (enemy in daftarMusuh)
    {
        danger += BahayaBerdasarkanJarakMusuh(enemy, posisi);
    }

    foreach (bullet in daftarVirtualBullet)
    {

```

```

    danger += BahayaBerdasarkanLintasanPeluru(bullet, posisi);
}

danger += PenaltiJikaDekatDinding(posisi);
danger += PenaltiJikaDekatPojoek(posisi);

return danger;
}
}

```

4.2 Struktur Data, Fungsi, dan Prosedur pada Bot Utama

Bot utama yang dipilih adalah MinimumDangerBot. Bot ini menggunakan strategi Greedy Minimum Danger, yaitu memilih posisi dengan nilai bahaya paling kecil pada setiap kondisi permainan. Strategi ini bertujuan agar bot dapat bertahan lebih lama, menghindari serangan musuh, dan tetap memiliki peluang untuk menyerang.

Struktur data utama yang digunakan adalah Dictionary<int, EnemyData> enemyData. Struktur ini digunakan untuk menyimpan data setiap musuh berdasarkan ID musuh. Data yang disimpan meliputi posisi terakhir musuh, energi musuh, arah gerak, kecepatan, status hidup, serta riwayat pergerakan musuh. Selain itu, bot juga menggunakan List<Bullet> bullets untuk menyimpan peluru virtual musuh dan List<MyBullet> myBullets untuk menyimpan peluru virtual milik bot.

Fungsi dan prosedur penting pada MinimumDangerBot adalah sebagai berikut:

Fungsi/Prosedur	Kegunaan
Run()	Mengatur konfigurasi awal bot, seperti warna, radar, dan inisialisasi variabel utama
OnTick()	Mengevaluasi posisi secara berkala dan memilih posisi dengan nilai bahaya paling kecil
OnScannedBot()	Memperbarui data musuh, mengunci radar, menentukan fire power, dan melakukan prediksi tembakan
CalcGrav()	Menghitung nilai bahaya suatu posisi berdasarkan musuh, peluru virtual, dan posisi pojok arena
AddVirtualBullet()	Menambahkan peluru virtual musuh ketika musuh diduga menembak
AddLinearVirtualBullet()	Menambahkan prediksi peluru musuh berdasarkan arah gerak linear

AddMyVirtualBullet()	Menyimpan data peluru virtual milik bot untuk mengevaluasi efektivitas tembakan
OnBulletFired()	Mencatat peluru yang ditembakkan bot sebagai peluru virtual
OnHitByBullet()	Mencatat ketika bot terkena peluru untuk menyesuaikan strategi dodge
OnBotDeath()	Menandai musuh yang sudah mati dan memperbarui target

Fungsi utama dalam strategi greedy bot ini adalah CalcGrav(). Fungsi tersebut digunakan untuk menghitung nilai bahaya dari beberapa kandidat posisi. Posisi dengan nilai bahaya paling kecil akan dipilih sebagai tujuan movement bot. Nilai bahaya dihitung berdasarkan jarak terhadap musuh, lintasan peluru virtual, jarak terhadap pojok arena, dan posisi terakhir musuh. Dengan cara ini, MinimumDangerBot dapat memilih posisi yang lebih aman dan meningkatkan peluang bertahan hidup selama pertandingan

4.3 Pengujian

4.3.1 Pengujian Internal Empat Alternatif Bot

Pengujian 1 :

Results for 100 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Minimum Danger Bot ...	1192	500	90	525	75	2	0	3	1	1
2	Range Keeper Greedy...	786	250	0	462	42	8	23	0	3	1
3	Aegis Greedy 4.0	711	200	30	423	48	10	0	2	0	2
4	Lock Bot 6.0	251	250	0	0	0	1	0	0	1	1

Pengujian 2:

Results for 100 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Range Keeper Greedy...	1184	400	60	573	98	53	0	2	1	0
2	Minimum Danger Bot ...	726	450	30	222	24	0	0	1	3	0
3	Aegis Greedy 4.0	467	200	0	247	17	2	0	0	0	4
4	Lock Bot 6.0	232	100	0	14	0	101	17	1	0	0

Pengujian 3:

Results for 100 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Minimum Danger Bot ...	977	500	60	381	35	0	0	1	3	0
2	Range Keeper Greedy...	864	300	30	474	51	8	0	2	0	2
3	Aegis Greedy 4.0	690	200	0	429	37	23	0	1	1	1
4	Lock Bot 6.0	162	150	0	0	0	12	0	0	0	1

Pengujian 4:

Results for 100 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Range Keeper Greedy...	746	300	30	354	49	12	0	3	0	1
2	Minimum Danger Bot ...	703	350	60	255	18	1	18	1	1	2
3	Aegis Greedy 4.0	409	150	0	232	27	0	0	0	2	1
4	Lock Bot 6.0	110	100	0	0	0	10	0	0	1	0

Pengujian 5:

Results for 100 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Minimum Danger Bot ...	1291	600	60	533	52	46	0	2	4	0
2	Range Keeper Greedy...	1185	350	60	555	91	128	0	2	0	2
3	Aegis Greedy 4.0	982	400	30	457	52	42	0	2	2	2
4	Lock Bot 6.0	183	150	0	7	0	26	0	0	0	2

4.3.2 Pengujian Bot Utama Melawan Bot Asisten

Pengujian 1:

Results for 100 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Spin Bot 2.0	1310	500	100	544	77	59	30	2	0	0
2	Minimum Danger Bot ...	840	400	0	319	17	103	0	1	0	2
3	Ram Fire 1.0	608	150	0	222	0	148	89	0	1	1
4	Fire 1.0	607	300	0	284	18	5	0	0	1	0
5	Painting Bot 1.0	577	500	0	68	8	0	0	0	1	0
6	My First Bot 1.0	454	350	0	100	0	4	0	0	0	0

Pengujian 2:

Results for 100 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Spin Bot 2.0	766	200	0	432	34	71	29	4	0	0
2	Minimum Danger Bot ...	533	350	60	119	4	0	0	0	4	0
3	Ram Fire 1.0	171	0	0	128	0	43	0	0	0	4

Pengujian 3:

Results for 100 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Minimum Danger Bot ...	498	200	40	213	36	8	0	5	0	0
2	Spin Bot 2.0	16	0	0	16	0	0	0	0	5	0

4.4 Analisis Hasil Pengujian

Berdasarkan hasil pengujian, terdapat dua jenis pengujian yang dilakukan, yaitu pengujian internal empat alternatif bot dan pengujian bot utama melawan bot asisten. Pengujian internal dilakukan untuk menentukan bot terbaik dari empat alternatif, yaitu MinimumDangerBot, RangeKeeperGreedy, AegisGreedy, dan LockBot.

Pada pengujian internal, MinimumDangerBot dan RangeKeeperGreedy menunjukkan performa paling baik. MinimumDangerBot memperoleh peringkat pertama pada pengujian 1 dan 3, sedangkan RangeKeeperGreedy memperoleh peringkat pertama pada pengujian 2 dan 4. Jika dilihat dari rata-rata skor, MinimumDangerBot memperoleh rata-rata sekitar 899,5, sedangkan RangeKeeperGreedy sekitar 895. Karena memiliki rata-rata skor sedikit lebih tinggi dan performa yang stabil, MinimumDangerBot dipilih sebagai bot utama.

Selanjutnya, MinimumDangerBot diuji melawan bot asisten atau bot pembanding. Pada pengujian pertama, MinimumDangerBot memperoleh skor 840 dan berada di peringkat kedua. Pada pengujian kedua, MinimumDangerBot memperoleh skor 533 dan kembali berada di peringkat kedua. Pada

pengujian ketiga, MinimumDangerBot berhasil menempati peringkat pertama dengan skor 498 melawan Spin Bot 2.0.

Hasil tersebut menunjukkan bahwa strategi Greedy Minimum Danger cukup efektif, terutama dalam meningkatkan kemampuan bertahan hidup. Bot mampu memilih posisi yang lebih aman berdasarkan nilai bahaya dari posisi musuh, peluru virtual, dinding, dan pojok arena. Namun, MinimumDangerBot masih perlu ditingkatkan pada bagian akurasi tembakan dan efektivitas serangan, karena pada beberapa pengujian masih kalah dari bot yang mampu menghasilkan damage lebih besar.

BAB V KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan implementasi dan pengujian yang telah dilakukan, algoritma greedy dapat diterapkan dengan baik pada bot Robocode Tank Royale. Algoritma greedy digunakan untuk menentukan movement, memilih target, mengatur radar, menentukan fire power, memprediksi gerakan musuh, dan menghindari peluru.

Empat alternatif bot yang dikembangkan memiliki strategi greedy yang berbeda. RangeKeeperGreedy menggunakan strategi Greedy Ideal Distance untuk menjaga jarak ideal terhadap musuh. LockBot menggunakan strategi Greedy Candidate Position Scoring untuk memilih kandidat posisi dengan skor terbaik. AegisGreedy menggunakan strategi Greedy Hybrid Target Scoring untuk memilih target berdasarkan skor tertentu. MinimumDangerBot menggunakan strategi Greedy Minimum Danger untuk memilih posisi dengan nilai bahaya paling kecil.

Berdasarkan pengujian internal, MinimumDangerBot memperoleh rata-rata skor tertinggi, yaitu sekitar 899,5, sedikit lebih tinggi dibandingkan RangeKeeperGreedy yang memperoleh rata-rata sekitar 895. Oleh karena itu, MinimumDangerBot dipilih sebagai bot utama karena memiliki performa yang paling stabil dan mampu bertahan dengan baik.

Hasil pengujian melawan bot asisten juga menunjukkan bahwa MinimumDangerBot mampu bersaing dengan bot pembanding. Pada pengujian pertama dan kedua, MinimumDangerBot menempati peringkat kedua. Pada pengujian ketiga, MinimumDangerBot berhasil menang dalam kondisi 1 vs 1 melawan Spin Bot 2.0. Hal ini membuktikan bahwa strategi Greedy Minimum Danger cukup efektif, terutama dalam kondisi duel.

Meskipun demikian, MinimumDangerBot masih memiliki kelemahan. Bot ini sudah cukup baik dalam bertahan hidup, tetapi belum selalu menghasilkan damage paling besar. Oleh karena itu, strategi greedy yang digunakan sudah berhasil meningkatkan survivability, tetapi masih perlu dikembangkan agar lebih kuat dalam menyerang dan memperoleh skor akhir yang lebih tinggi.

5.2 Saran

Pengembangan selanjutnya sebaiknya difokuskan pada peningkatan akurasi tembakan dan pengaturan fire power agar damage yang diberikan lebih besar. Selain itu, parameter perhitungan bahaya seperti jarak musuh, peluru virtual, dinding, dan pojok arena dapat disesuaikan lagi agar bot tidak hanya aman, tetapi juga tetap efektif menyerang. Pengujian juga sebaiknya dilakukan lebih banyak dalam format 1 vs 1 terhadap berbagai bot asisten agar performa MinimumDangerBot dapat dianalisis lebih akurat

LAMPIRAN

Lampiran 1. Tautan Repository GitHub

Repository GitHub:

https://github.com/HafidzHqq/Tubes_GATOT.git

Lampiran 2. Video Demo

Video demo:

<https://youtu.be/6n-z9gREyyI>

DAFTAR PUSTAKA

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

Munir, R. (2005). *Diktat Kuliah Strategi Algoritma*. Institut Teknologi Bandung.

Robocode Tank Royale. (2024). *Robocode Tank Royale Documentation*. Robocode Tank Royale.

Robocode Tank Royale. (2024). *Robocode Tank Royale Bot API Documentation*. Robocode Tank Royale.

Microsoft. (2024). *C# Documentation*. Microsoft Learn.

Microsoft. (2024). *.NET Documentation*. Microsoft Learn.

Tim Asisten Strategi Algoritma. (2026). *Spesifikasi Tugas Besar Strategi Algoritma: Robocode Tank Royale*. Program Studi Teknik Informatika.

Kelompok 9 GATOT. (2026). *Source Code Bot Robocode Tank Royale: MinimumDangerBot, AegisGreedy, LockBot, dan RangeKeeperGreedy*. Institut Teknologi Sumatera.